PART III — BUILDING DIGITAL PRODUCTS · CHAPTER 10

From the Factory Floor to Code

10

How a digital product gets built: organizing work to produce with quality, adapting it when we don't yet know the solution, and confirming the result serves the person who receives it.

Nov 11, 2026 · 24 min read · Living edition 0.10 · Status: in progress ·

A bottle can leave the line in perfect condition and end up in the trash because nobody could buy it a replacement cap. Between those two moments sits an entire business. It also holds a way of understanding how a digital product gets built.

Before talking about methodologies, it's worth being clear about what we're actually trying to do. A product is not just an object or an application: it's a solution someone uses to solve a need. Manufacturing it means transforming materials and coordinating people; building its digital version means organizing information, decisions and programs. In both cases we can follow every instruction to the letter and still fail the person who needed it.

Over this chapter we'll work through three questions. How is work organized to produce with quality? How does that way of working change when we don't yet know the solution? And how do we know the result serves the customer once it reaches them? Each question gave rise to practices and methodologies with proper names of their own.

There's no need to know them in advance — they'll show up once the problem they're meant to solve is already on the table.

A Cap on the Table



FIGURE 10.1 Six bottles, six caps: what the manufacturer records as six well-listed items, the customer experiences as one part they may not be able to buy.

Picture a factory that makes insulated bottles. At the start of the shift there are steel bodies, caps, silicone gaskets and cardboard boxes. An order specifies which model to build and how many units are needed. Someone has to turn that order into a sequence that people with different tools can execute, check and repeat.

The body is formed, welded and checked for leaks. At another station, the gasket is seated inside the cap. The two parts come together in assembly, pass through a leak test and receive a lot identification before packing. The tolerance the designer noted on a drawing now has a fairly intimate consequence for the customer who carries the bottle alongside their laptop.

Months later, that customer loses the cap. They go to the manufacturer's site and find six nearly identical photographs. The thread changed between two generations, but both bottles still carry the same commercial name. From the manufacturer's point of view, there are six well-listed items. From the customer's point of view, there's a bottle they still won't be able to use tomorrow. We're going to follow that problem from the workshop to the screen.

Before measuring the speed of the line, it's worth separating two activities that look alike: **doing work** and **moving a product forward**. A person can assemble caps without stopping and still see no bottle come out, because the next station is saturated. When we talk about *flow*, we mean that entire journey, from the moment work enters to the moment it delivers something usable. Watching only how busy each person looks can hide where the whole system actually stalls.

The Factory's Rhythm



Say we have 420 usable minutes per shift and orders for 210 bottles. Dividing the available time by the units requested, we find we need to finish one every two minutes. Production systems call that required rhythm *takt time*, a term borrowed from the German *Takt*, meaning beat. If the leak test takes three minutes per bottle at a single station, that station can process at most 140 during the shift, even running without interruption. Speeding up assembly will only grow the line in front of the test.

Problems like this pushed a different way of thinking about production. Over the twentieth century, Toyota developed a system aimed at building only what was needed, catching failures



Figure 10.2 — Takt time marks how often a finished bottle should come out, not how long one person takes to make it.

early and learning from everyday work without relying solely on large inventories or end-of-line inspection. Decades later, researchers and practitioners gathered several of those ideas under the name **Lean**. Its purpose isn't to ask people to work faster: it's to identify which steps produce value for the customer and remove the obstacles that consume time and resources without improving the result.

That's how a Lean-inspired intervention would start. It would go to where the work happens, not only to the room where indicators get presented. Toyota uses the Japanese word *gemba* for this: the real place where we can check what's going on. It would follow one batch from arrival to departure, note transformation time and waiting time separately, and put the adapters within reach of whoever runs the test. A standard exists to compare two ways of working and keep the one that performs better.

The **Toyota Production System**, Lean's central antecedent, rests on two complementary principles. The first is **just-in-time**: producing and replenishing at the moment and in the quantity needed, rather than filling the factory with parts in case they're needed someday. The second is **jidoka**: building in the ability to recognize a problem and stop the process rather than keep making defects. With *just-in-time*, the consumption of caps in assembly signals when to replenish them; a *kanban* card — the Japanese word for a sign or visual signal — can carry that need. With *jidoka*, if poorly seated gaskets show up, the station can stop and call for help before packing defective bottles.[1]

2 min	140	8
TAKT TIME FOR THE SHIFT (420 MIN / 210 BOTTLES)	MAXIMUM UNITS THROUGH THE LEAK TEST PER SHIFT	STEPS IN TOYOTA'S PROBLEM-SOLVING METHOD

Naming the Waste



Figure 10.3 — Inventory taking up a corridor: one of the seven classic forms of muda.

To know what's getting in the way of flow, we need a way to name it. In Lean thinking, *muda* is a Japanese word for work or resource consumption that adds no value from the customer's point of view. An inspection genuinely required for safety or quality doesn't vanish by decree: we first have to separate what's indispensable from what's avoidable.

In a single morning we could find all seven classic forms. There's **overproduction** when caps nobody ordered get made, **inventory** when they take up a corridor, **transport** when the batch crosses the floor twice, and **motion** when someone has to bend down for every adapter. There's **waiting** in front of the test, **overprocessing** when an already-recorded number gets typed in again, and **defects** when a cap leaks. An eighth waste later became popular: **unused talent**. The operator who knows why the

gasket twists has spent three months straightening it by hand. When do we ask her what she'd change?[2]

Eight Steps to Fix a Leak



Figure 10.4 — Naming waste isn't enough to fix it; you first have to describe exactly what's failing.

Naming waste isn't enough to fix it. If six bottles out of a hundred leak, we could blame a person, adjust the machine with no data, or investigate before acting. Toyota developed a practical problem-solving discipline that starts by describing exactly what's failing and ends by checking whether the fix holds. Its **eight steps** aren't a ceremony: they stop us from jumping from annoyance to a favorite solution without understanding the cause.

Waste helps us look. The eight steps help us intervene. Here's how they'd play out in a workshop exercise, with illustrative figures.[3]

TABLE 10.1 — EIGHT STEPS TO FIX A LEAK

STEP	WHAT WE'D DO ABOUT THE CAP
1. Clarify the problem	Out of every 100 bottles tested, six leak. Identify how the test was run and what counted as passing.
2. Break it down	Split results by model, gasket batch, shift and station to find where the failure concentrates.
3. Set a target	Bring the affected model's leak rate to one per 100 units or fewer within four weeks.

STEP	WHAT WE'D DO ABOUT THE CAP
4. Investigate the root cause	Watch the assembly and check whether the gasket shifts as the cap is tightened. Reproduce the failure before blaming the operator.
5. Design countermeasures	Compare an assembly guide against a change to the gasket housing. Choose what to try first, and why.
6. Apply the countermeasure	Trial the guide at one station and log which batches and shifts use it.
7. Evaluate process and result	Compare leak rates under equivalent conditions and check whether assembly time went up.
8. Standardize and share	If the improvement holds, update the method, train the other shifts and verify their performance.

Sometimes finding the cause takes more than observation. Two batches made with the same instructions can behave differently because of small variations in materials, measurements or machines. **Six Sigma** is a quality-improvement approach developed at Motorola in the 1980s that uses data and statistical tools to reduce defects and variation. It doesn't replace Lean: it can complement the observation of flow when we need to tell real patterns apart from chance.

One of its working paths is called **DMAIC**, for *Define, Measure, Analyze, Improve, Control*. With DMAIC we define what counts as a leak, measure with a reliable instrument, analyze the variation, test an improvement and leave in place a control that flags when the process drifts again. Two shifts can apply the same average torque and get different results if one has far more spread. Changing the specification based on the average alone could make both of them worse.[4]



Figure 10.5 — The tolerance a designer noted on a drawing becomes an intimate consequence for whoever uses the bottle every day. Conceptual view.

Lean stays relevant because the gap between leadership's intent and what the last station can actually deliver reopens with every product.

Local improvement isn't enough when design, purchasing, sales and operations work from contradictory priorities: a factory can do perfectly what the company decided badly, such as making bottles

without planning for cap resupply. To line up those levels, Japanese organizations adopted **Hoshin Kanri**, a goal-deployment practice that connects strategic decisions to concrete plans and lets commitments between areas be reviewed. If sales proposes offering caps for ten models and purchasing can only secure four, that conversation has to happen before the promise gets printed on the box.[5]

How the Idea Reached Software

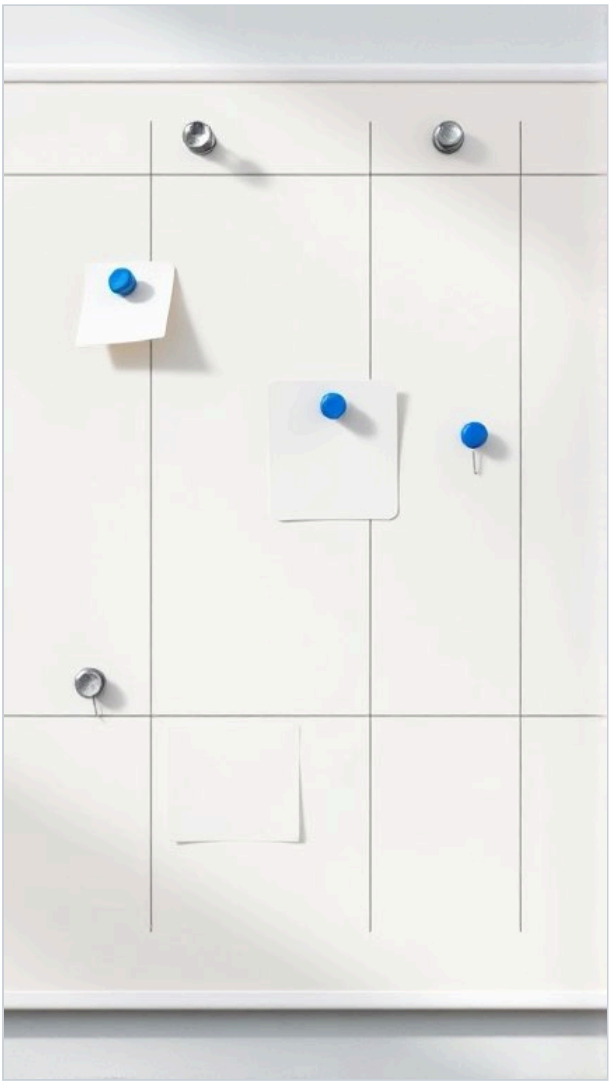


Figure 10.6 — Teams moving forward together, with overlapping activities instead of sequential stages.

In the factory, we know the bottle's design and want to produce many identical units. What happens when we're still discovering how the product should work? We can't ask a team to repeat, a hundred times over, a solution it hasn't found yet.

In traditional software development it was common to organize work into successive stages: first all requirements were defined, then design happened, then coding, then testing. That separation made it easier to plan responsibilities, but it could let interpretation errors surface very late. Out of that difficulty came practices favoring early collaboration, partial deliveries and the chance to correct course based on real results. Over time these got grouped under the idea of **agility**: adapting to what's learned without losing the discipline of shipping.

In 1986, Hirotaka Takeuchi and Ikujiro Nonaka described product-development teams that moved forward together, with overlapping activities; the image they chose was a rugby team. That industrial observation later inspired **Scrum**, a framework in which a team collaborates over short periods, shows a concrete result and adjusts its plan based on what it learned. In 2001, the signatories of the **Agile Manifesto** put into words values shared across several strands of software development, including Scrum, Extreme Programming and Crystal: collaboration, working software, customer relationship and responding to change.[6, 7]

What we can carry over from the factory is the ability to see accumulated work, shorten the time between an error and its detection, and give the team the authority to fix it. What changes is the nature of the unit being produced. In the workshop we want bottle number two hundred and ten to keep the properties of the first; in software, a single version of the program is usually enough to serve many buyers.

1986	2001	MVP
TAKEUCHI AND NONAKA DESCRIBE THE RUGBY TEAM	THE AGILE MANIFESTO IS SIGNED	MINIMUM VIABLE PRODUCT, NOT A BAD PRODUCT

The Minimum Viable Product



Figure 10.7 — Before funding a smart catalog, print the code on a batch of packages and fulfill the right cap by hand.

But shipping software frequently doesn't guarantee anyone wants to use it. We can build a flawless cap finder that nobody buys because shipping costs more than a new bottle. **Lean Startup**, an approach popularized by Eric Ries drawing on entrepreneurship experience and validated learning, answers that uncertainty: before investing heavily, form a testable assumption, build the smallest test that lets you learn, and decide with evidence whether to continue or change course. That initial test is usually called a **minimum viable product**, or *MVP*; it doesn't mean a bad product, but the least you need to check a hypothesis with real users.[8]

Our first hypothesis might be that people would buy the replacement part if they knew how to identify it. Before funding a smart catalog, we can print the model code on a batch of packages and offer manual help to fulfill the right cap. We'd count requests, purchases, errors and the cost of support. If most people identify the part but drop off once they see the shipping cost, we've just changed the decision about what to build. That's what building, measuring and learning means in practice.

The Journey of a Change



Figure 10.8 — Opening a branch: a line of work that changes the search tool without yet touching the shared version.

For a product designer, the flow is usually visible up to the point the screens are handed off. After that, a card appears saying “in development,” and several days go by. The change we’re interested in is small to state: a person types the code stamped on the base of their bottle, sees the compatible cap, and can order it. Making that happen requires the interface, the catalog, inventory and the order-taking system to talk to each other.

Code is the set of instructions a program runs. A **repository** is a shared place where that code and its history of changes live, like a log that lets you recover earlier versions. And a **version-control system**, like Git, records who proposed each change and helps combine several people’s work without accidentally overwriting it. This isn’t a factory shipping more bottles: every change rewrites the instructions we’ll use to serve every buyer.

Design discovers the stamped code is hard to read and adds an illustration showing where to look for it. Engineering asks what should happen if someone mistakes a zero for the letter O. The person responsible for **quality assurance**, or *QA*, proposes testing a bottle whose code still exists even though its cap has left the catalog. These questions reshape the design while moving a decision is still cheap.

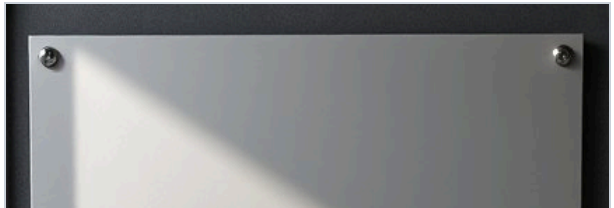
TABLE 10.2 — WHAT YOU HEAR ON THE TEAM

WHAT YOU HEAR	WHAT JUST HAPPENED
“I opened a branch”	A line of work opens in the repository to change the search tool without yet touching the shared version.
“I made a commit”	The developer records an identifiable change locally and explains the intent behind it.
“I pushed it”	Their commits went to the remote repository. Colleagues can see them, and configured checks can run.
“I opened a pull request”	They’re asking for the branch to be reviewed and merged. The team sees the diff, test results and open questions.
“It got merged”	The changes are now integrated into the target branch.
“It’s deployed”	An identified version was installed into an environment, production for example.

How often should you push? Whenever there’s a coherent change to share, a review worth asking for, or a point of collaboration worth preserving. As a starting agreement for a team working with continuous integration, I’d aim to merge small changes into the main branch at least daily, with tests passing. A daily push to an isolated branch for three weeks still leaves the integration problem untouched.[9, 10]

The next problem shows up when integrating one change can break another that already worked. To catch that earlier, teams began automating the build and the tests every time changes came in. That’s where the practice of **continuous integration** comes from.

CI/CD, DevOps and DevSecOps



Continuous integration (CI) pulls those changes together often and runs automatic checks. When someone edits the catalog, the system builds the



Figure 10.9 — Automatic checks every time a change lands in the shared repository.

program and verifies, among other things, that earlier codes still return their cap. **Continuous delivery (CD)** keeps a verified version ready to deploy whenever the team decides. In **continuous deployment**, changes that clear the established checks reach production automatically.

An application isn't finished once someone writes the code. It also has to stay available, protect data and respond to failures. The sustained collaboration between development and operations came to be called **DevOps**; making security explicit throughout that same process is **DevSecOps**.

Checking compatibility only needs the bottle's code. Asking the buyer for their email on that screen would add data to safeguard without helping them find the cap. That's a security-in-design conversation. In the repository, permissions and dependencies get reviewed; in production, errors are monitored and it's known who can turn off the search tool if it starts mixing up models.[11]

What the Delivery Lead Does Next



Here a responsibility shows up that's different from designing screens or coding features:



Figure 10.10 — Agreeing to fulfill the first orders by hand, if operations can manage it and the customer knows the timeline.

getting several people's work to turn into a capability the customer can actually use. The person who coordinates scope, dates, budget, risks and dependencies on a project is often called a **Project Manager (PM)**; when people talk about *delivery*, they're emphasizing actual delivery, not just activities completed.

One costly way to organize this product would be to ship all six screens first, then every service, and only at the end connect inventory — weeks could pass with three teams reporting progress without having sold a single cap. Choosing one family of bottles and carrying its journey through end to end, until someone receives the correct part, accepting that the first orders might be fulfilled by hand if operations can manage it, is what **adding value iteratively** looks like. The first increment recovers use of a bottle; the second stops charging for a cap that's out of stock.

In a delivery review, it's worth looking at one order end to end: how many days the oldest unintegrated change has been sitting, how long a review has been waiting, what dependency is blocking a test against inventory, and which defects have reopened.

A board with five cards stuck in front of the one person who can

approve the catalog looks a lot like a line of bottles waiting for the adapter.

Opening five more stories would just keep the team busy. Getting that adapter would change the delivery date. The bottleneck is rarely how many tasks we've opened — it's who can actually clear them.

The Bug That Survives the Demo

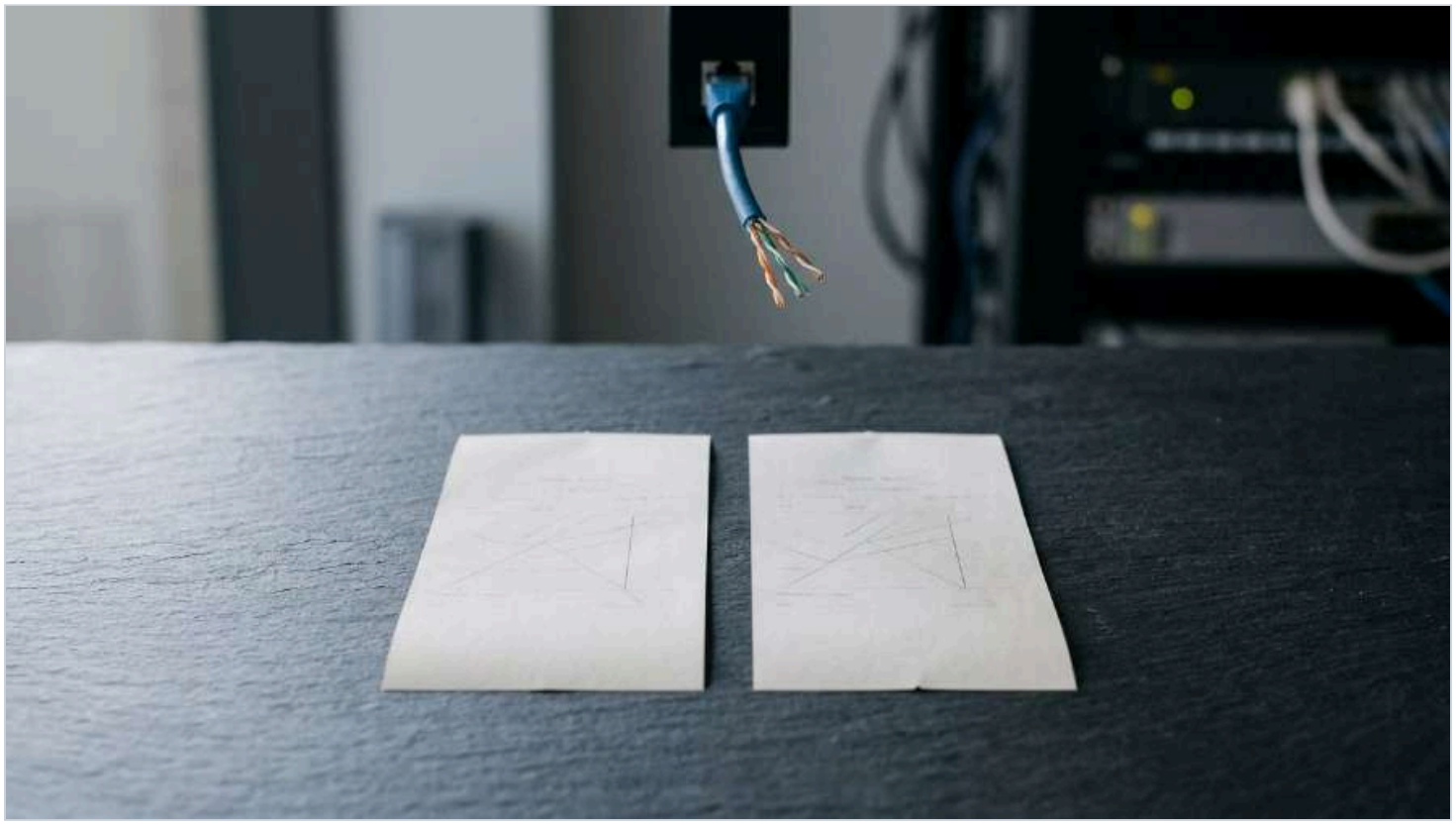


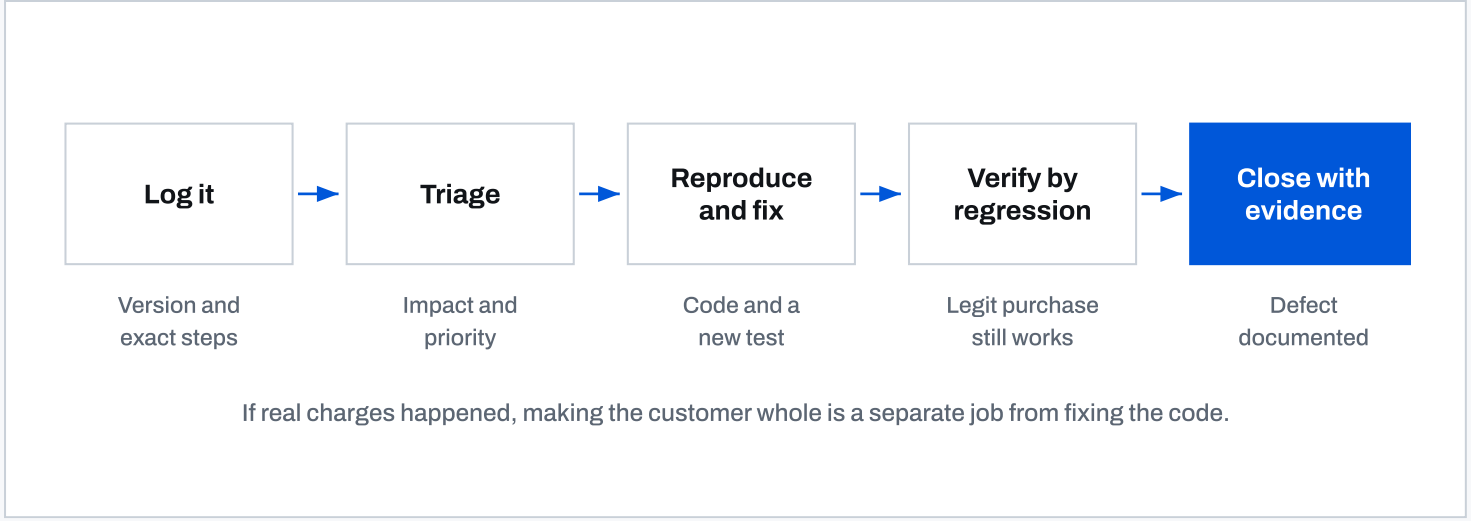
FIGURE 10.11 QA cuts the connection right after confirming. The customer retries, and two orders appear.

In the demo, we type a code, pick a cap and watch the order confirm. QA does something less friendly and much closer to real life: it cuts the connection right after confirming. The customer retries, and two orders appear. The screen looks exactly as design approved it. The customer just paid twice.

The fix cycle starts by recording version, environment, steps, expected result and evidence of the double charge. In an early review called *triage* — a term borrowed from medical urgency classification — quality, engineering and product weigh impact, scope and priority. Engineering reproduces the failure, fixes the retry handling and adds a test that tries to duplicate the order. QA repeats the case on the new version and checks, through regression, that a second legitimate purchase is still possible. If real charges already happened, operations also needs to identify who was affected and make it right: fixing code and refunding money are two different jobs.

Up to now we've asked whether the program works technically. We still need to know whether it does the actual job. **UAT** stands for *User Acceptance Testing*: business users checking whether they can complete representative situations with the expected outcome. Many teams involve a **Product Owner**, the person who prioritizes product needs and agrees with the business on what counts as an acceptable result. If the storefront promises three days and the warehouse needs eight, the technical tests can be all green while the product still isn't ready to announce.

Figure 10.12



Fixing the code and refunding the money are two different jobs, even though both start from the same defect.

Listening to the Customer, Measuring the Effect



Not every question gets answered by a software test. To understand why someone picks the wrong cap, we can start by listening to them. A **focus group** brings participants together to talk and watch reactions to an idea, object or experience. It's useful for surfacing language, motivations and points of confusion; on its own, it can't tell us what the whole population will do.

Put a bottle and two packages on the table. One person turns the bottle over looking for the model on the base; another reads "series" on the box and copies down the wrong number; someone suggests printing a silhouette of the thread. Four participants preferring the



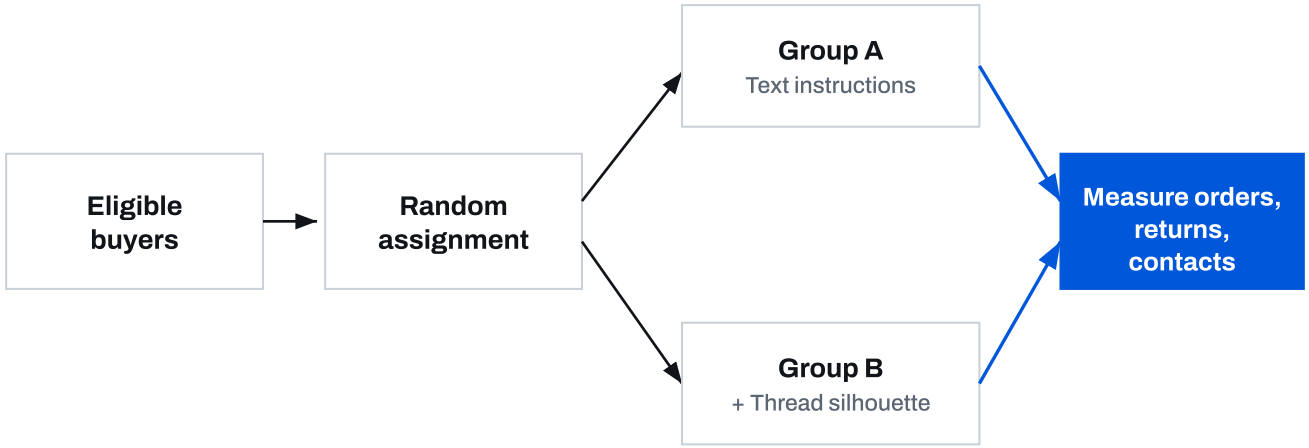
Figure 10.13 — Ask participants to find the replacement part and explain what they're looking at.

silhouette still leaves open how many correct purchases it would actually produce.

In digital, we can watch usage sessions with the same closeness and then test a hypothesis at scale. An **A/B test** is a comparative experiment: two similar groups get different versions, and we measure which produces a better result on the thing we care about. Random assignment aims to keep one version from looking better purely because different people used it. A sees the text instructions, B adds the silhouette; we agree beforehand to measure compatible orders per assigned visitor and to watch returns, cancellations and support contacts.

If traffic is barely enough to sell ten caps a week, a statistical comparison can take longer than the decision is worth. In that case I'd rather watch buyers directly, follow orders, and be honest about how much is still left to learn.[12]

Figure 10.14



Minimum effect, sample size and observation window are set before looking at results.

Checking results on day one and calling the version with more clicks the winner can reward the illustration that confuses people most.

What It Costs to Reach the Market

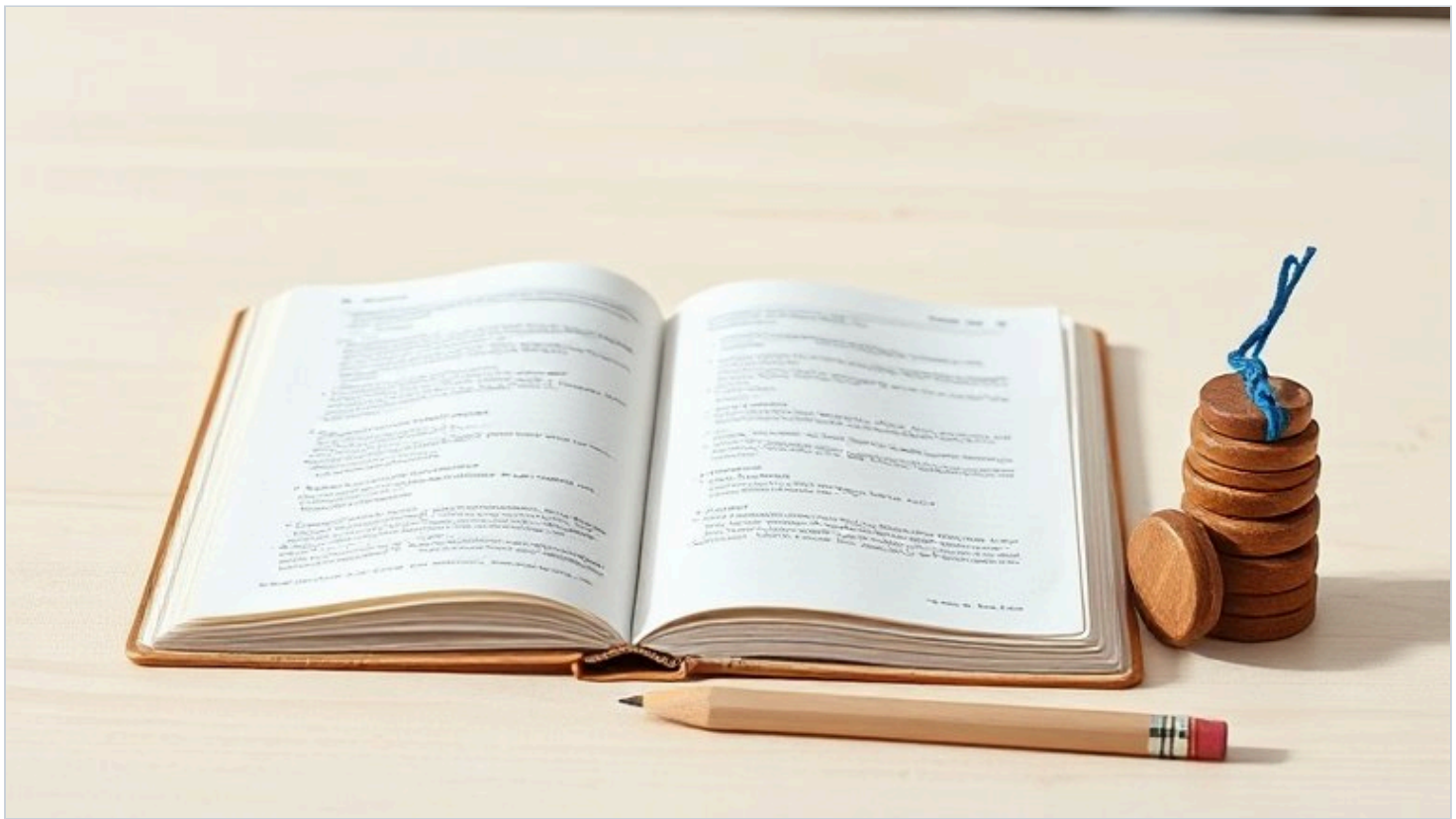


FIGURE 10.15 Setting aside a share for launch and the first iterations, and clearing the rest before opening to the public.

The factory knows the difference between approving a sample and authorizing a full lot. In digital, we also need to separate the version that works from the decision to put it in front of thousands of people. The following conversation should happen before buying any advertising, while the promise can still be changed.

TABLE 10.3 — PRE-LAUNCH DECISION

PRE-LAUNCH DECISION	BOTTLE AND REPLACEMENT PART	DIGITAL SERVICE
What's promised, and to whom	Compatible models, price, warranty, distribution channels	Enabled segments, stock by region, total price and timeline shown upfront
What proves it can be delivered	Pilot batch, verified leak-tightness and assembly	Full order tested, UAT, monitoring and recovery rehearsed
How much volume it can serve	Capacity, safety stock and resupply	Platform capacity, synced inventory, and people to fulfill and support
What gets communicated	Packaging, instructions, store and distributor materials	Search page, email, campaign, support script and activation date
Who decides to stop	Quality lead freezes the lot and traces units	On-call owner disables the feature, keeps traceability and coordinates support

Building and launching are also economic decisions. **Budget** is the money authorized for a purpose; **actual spend** is what's already been used; **commitments** are obligations taken on even if not yet paid; and **forecast** is an estimate of what we'll end up spending. Mixing those up can make a project that's already promised more resources than it has look perfectly healthy.

I'd put a budget behind these decisions. Say, for this exercise, an approved ceiling of **one million Mexican pesos**. We set aside **400 thousand** for launch and the first iterations, and authorize up to **600 thousand** before opening to the public. A line that needs new molds would have a different split; either way, the reserved amount has to cover concrete obligations: distribution, media, support, measurement and time to fix a first version.

If 480 thousand has already been spent, there are 70 thousand in committed orders still unpaid, and we estimate another 90 thousand in work not yet contracted, the projected cost before launch is **640 thousand**. Even though treasury has only paid out 480 thousand, we're already projecting we'll go **40 thousand** over the limit. The PM needs to bring that decision forward now: defer a color variant if that frees the 40 thousand without hurting the pilot's promise, request an evidence-backed increase, or pause the launch if what's pending threatens reliability.

Forecasting sales isn't enough either. If each order leaves 80 pesos after manufacturing, subsidized shipping, fees, expected returns and variable support, paying 100 pesos to acquire a single purchase leaves a 20-peso shortfall. You can decide to fund that if there's evidence of repeat purchases. For a replacement part designed to last years, I'd want to see that evidence before signing off on the campaign.

$1M	$640K	$40K
APPROVED CEILING FOR THE EXAMPLE	PROJECTED COST BEFORE LAUNCH	OVER THE LIMIT, NOT YET SPENT

The Two Curves of Work

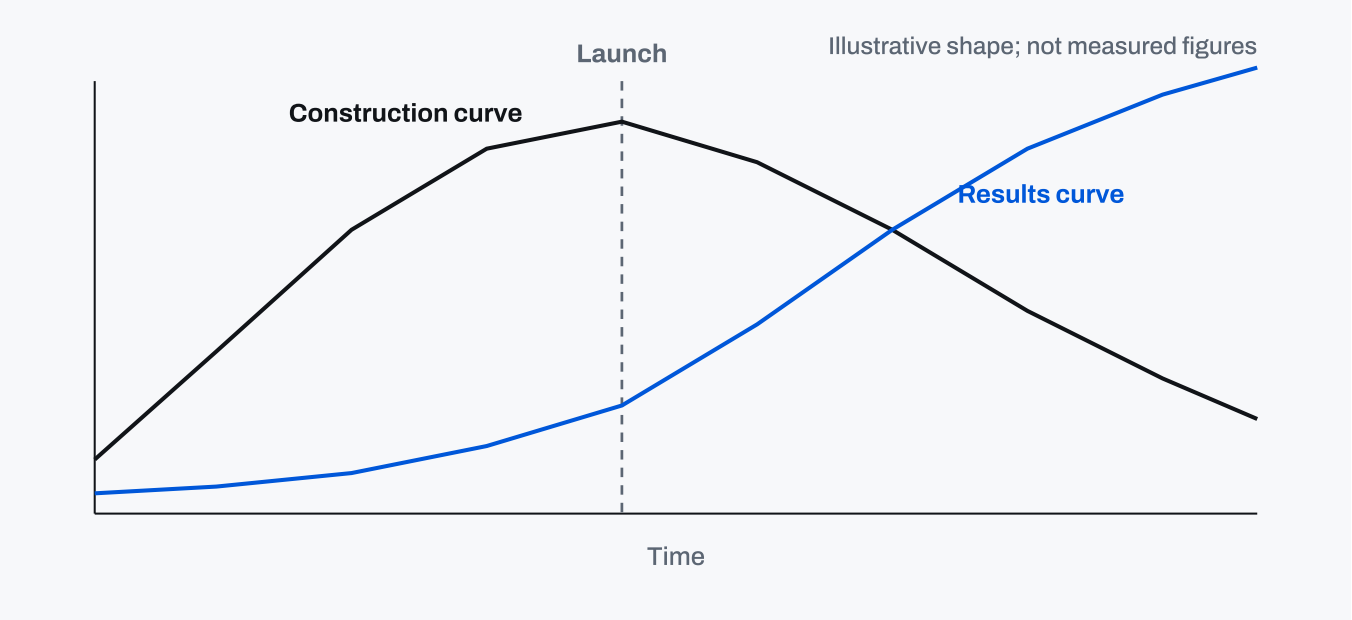


Figure 10.16 — A designer can generate twenty cap compositions in a morning; the responsibility grows at the point of choosing the right one.

For years it was easy to defend a construction budget with something you could point to: a mold, a machine, a finished screen. The team would grow to reach launch. The next morning, questions about sales, adoption and retention would start, sometimes with fewer people and the budget already spent. **Artificial intelligence (AI)** can cut part of the effort of producing materials and programs; it doesn't remove the work of finding out whether what got produced is correct, useful or sustainable. That's why I draw this kind of project with two effort curves.

The first curve starts before any code is written. It grows with observation, design decisions, architecture and testing. An agent can propose the cap finder and generate test cases the same afternoon; someone still has to discover that two threads share a name, get the right catalog, and decide how to handle a discontinued model. If that decision comes late, the tool can also help us implement the wrong answer faster.[13]

Figure 10.17



If the peak of results lands the same day as the peak of construction, we pull resources right when we can finally start learning from real buyers.

The second curve starts before opening to the public, when distribution, communication and instrumentation are being prepared; it can keep growing after launch and overtake the first. Now we have to find out why someone saw the ad, found the model, and dropped off once they saw the shipping cost. The result has its own calendar: impressions arrive right away, purchases later, and returns weeks after that.

A designer can generate twenty cap compositions in a morning; their responsibility grows the moment one of those images shows a thread the warehouse could never actually fulfill. Someone working in **growth** — measurably improving customer acquisition, activation and retention — can run more experiments with the same team, even if available traffic is still only enough to settle one question. Someone working in **SEO** (search engine optimization) can take the questions landing in support and turn them into a guide that helps identify an older model; generating hundreds of near-identical pages takes little time and still doesn't answer that question.[14]

I want the first iterations after launch to already be planned for at the design stage. The day we open the pilot, someone should know the cost per correct order, someone should be able to call the customer who returned a cap, and someone should have the authority

to change the ad or stop the sale. The money set aside after launch buys that ability to act while the problem still has a name and a face.



Figure 10.18 — I'd ask the operator who's spent three months straightening gaskets what she'd change about the mold.

My read on the digital market is an uncomfortable one: we're going to need more people who think and question, and fewer roles dedicated solely to executing instructions. I'd ask the operator who's spent three months straightening gaskets what she'd change about the mold. I'd ask the designer who can generate twenty ads which one she'd put in front of the buyer, and what result would make her pull it.

Both of them know something the director will rarely see from behind a desk. If we're going to invest in intelligence, I'd start by giving them the authority to use their own.

REFERENCES AND NOTES

The insulated-bottle factory and its figures are a didactic exercise built for this chapter, not the record of a real project. External references were consulted on November 9, 2026.

1. Toyota Motor Corporation, Toyota Production System. Just-in-time and jidoka. ↩
2. Lean Enterprise Institute, The Eight Wastes of Lean. ↩

3. Lean Enterprise Academy, Back to Basics: Problem Solving at Toyota. The eight-step problem-solving method. ↩
4. American Society for Quality, DMAIC Process. ↩
5. Lean Enterprise Institute, Hoshin Kanri. ↩
6. Hirotaka Takeuchi and Ikujiro Nonaka, The New New Product Development Game, Harvard Business Review, 1986. ↩
7. Jim Highsmith, History of the Agile Manifesto, 2001. ↩
8. Eric Ries, The Lean Startup Methodology. ↩
9. GitHub Docs, GitHub flow. ↩
10. DORA, Trunk-based development. ↩
11. NIST, Secure Software Development Framework, SP 800-218. ↩
12. Microsoft Research, Patterns of Trustworthy Experimentation. ↩
13. DORA, State of AI-assisted Software Development, 2025. ↩
14. Google Search Central, Guidance on AI-generated content for websites. ↩

Edition history — Chapter 10

Version	Date	Note
v0.1	Nov 11, 2026	First bilingual edition (EN/ES): from the factory floor to code, takt time and Lean, the eight steps, Hoshin Kanri, how the idea reached software, MVP, Git and CI/CD, DevOps, coordinating delivery, the bug that survives the demo, listening to the customer and A/B testing, what it costs to reach the market, the two curves of work.

← PREVIOUS CHAPTER
09 — Engineering Excellence as a Product Decision

NEXT CHAPTER →
11 — in development

© 2026 J. Angel Sosa. *Thinking in Products, Building Systems.*